Proje Bağlamı

Bu proje **in|sure Partner** sistemi için UI test otomasyonu içeren bir multi-module Maven projesidir.

Teknoloji Yığını

- Java 17 (JDK: `C:\java\jdk-17.0.18.8-hotspot`)
- Test Framework: TestNG
- Browser Automation: insure-automation-selenium (Playwright abstraction katmanı)
- Browser Provider: Playwright (config ile seçilebilir)
- Build Tool: Maven
- Target System: JSF tabanlı Partner uygulaması (`http://localhost:8080/partner`)

Modül Yapısı

`partner-automation` (Framework modülü)

- **Pages**: UI elementlerini temsil eden PageObject sınıfları
- **Fieldsets**: Yeniden kullanılabilir UI bileşenleri
- **Actions**: İş akışlarını yöneten sınıflar (Anmeldung, Suche, Erstanlage)
- **Data**: Fluent builder pattern'lı data transfer nesneleri

`partner-regressionstest` (Test modülü)

- partner-automation framework'ünü kullanır
- Arrange-Act-Assert (AAA) pattern zorunludur
- TestNG `@Test` anotasyonu kullanılır

Build Komutları

```bash
export JAVA_HOME="/c/java/jdk-17.0.18.8-hotspot"
mvn clean install -s ~build/settings.xml
mvn test -s ~build/settings.xml
```

Önemli Kısıtlamalar

- Test verileri sistemde kalır, otomatik temizleme yoktur
- Testlerin sıralı çalışma bağımlılığı vardır
- Duplicate kayıt handling gereklidir
- Maven settings: `~build/settings.xml`
- Internal Artifactory: adesso-insure.de (credentials: `ARTIFACTORY_CREDENTIALS_USR` / `ARTIFACTORY_CREDENTIALS_PSW` env variables)

Görevler

1. Proje Yapısını Analiz Et

- `documentation/` klasöründeki tüm dosyaları oku
- `CLAUDE.md` ve `FILES_REFERENCE.md` dosyalarını incele
- Mevcut Page, Fieldset ve Actions sınıflarının pattern'ını anla
- `pom.xml` dosyalarını (root + her modül) incele

2. Fieldset / Page Object Üretim Standardını Belirle

Mevcut sınıfları inceledikten sonra şu soruları yanıtla:

- Locator stratejisi nedir? (ID > Name > CSS > XPath sırası)
- Fieldset sınıfları hangi base class'tan extend ediyor?
- Fluent builder nasıl implement edilmiş?
- Actions sınıfları Page'lere nasıl bağlanıyor?

3. CLAUDE.md Güncelle

Aşağıdaki bölümleri ekle veya güncelle:

```markdown
```

```
## Fieldset Üretim Kuralları
- JSF component'lar için locator önceliği: By.id() > By.name() > By.cssSelector()
- Her Fieldset kendi UI bileşenini temsil eder, iş mantığı Actions'ta olur
- Fluent builder pattern zorunludur (Data sınıfları için)
- Actions sınıfında her iş akışı için ayrı metod oluştur

## Test Yazım Kuralları
- Zorunlu pattern: Arrange-Act-Assert (AAA)
- TestNG @Test anotasyonu kullan
- Test bağımlılıklarını @Test(dependsOnMethods=...) ile belirt
- Duplicate kayıt durumunu her testte handle et

## Kod Üretim Öncelikleri
1. Mevcut sınıfların pattern'ına %100 uy
2. Yeni Fieldset üretirken en benzer mevcut Fieldset'i örnek al
3. Actions sınıfı üretirken Anmeldung/Suche'yu referans al
```

4. Yeni Fieldset Üretimi (JSF Sayfaları için)

Bir JSF sayfasının HTML'i verildiğinde:

1. Tüm form elementlerini ve ID'lerini tespit et
2. Mevcut Fieldset pattern'ına uygun Java sınıfı üret
3. İlgili Actions metodunu yaz
4. Örnek TestNG test metodu oluştur

5. Mevcut Testleri Refactor Et

- AAA pattern'a uymayan testleri düzelt
- Magic string'leri constant veya data builder'a taşı
- Tekrar eden kod bloklarını Actions'a çıkar
- Eksik `dependsOnMethods` bağımlılıklarını ekle

6. Dokümantasyon Üret

Her yeni sınıf için:

- Javadoc yorum bloğu ekle
- `FILES_REFERENCE.md` dosyasını güncelle
- Varsa ilgili test senaryosunu dokümante et

Çalışma Tarzı

- Her değişiklikten önce mevcut kodu oku ve anla
- Pattern'dan sapma yapmadan önce sor
- Birden fazla dosya değiştireceksen önce listeyi göster, onay al
- JSF component ID'leri tahmin edilebilir olduğu için HTML snapshot üzerinden doğrudan locator çıkar
- Üretilen her sınıfı projenin mevcut naming convention'ına uygun adlandır